# 38. Приоритет операций и правила ассоциативности

Чтобы правильно вычислять выражения (например, `4 + 2 * 3`), мы должны знать, что делают определенные операторы и в каком порядке они выполняются. Последовательность, в которой они выполняются, называется **приоритетом операций**. Следуя обычным правилам математики (в которой умножение следует перед сложением), выражение, приведенное выше, обрабатывается следующим образом: `4 + (2 * 3) = 10`.

В языке C++ все операторы (операции) имеют свой уровень приоритета. Те, в которых он выше, выполняются первыми. В таблице, приведенной ниже, можно увидеть, что приоритет операций умножения и деления (5) выше, чем в операциях сложения и вычитания (6). Компилятор использует приоритет операторов для определения порядка обработки выражений.

А что делать, если у двух операторов в выражении одинаковый уровень приоритета, и они размещены рядом? Какую операцию компилятор выполнит первой? А здесь уже компилятор будет использовать **правила ассоциативности**, которые указывают направление выполнения операций: слева направо или справа налево. Например, в выражении `3 * 4 / 2`: операции умножения и деления имеют одинаковый уровень приоритета (5-й уровень). А ассоциативность пятого уровня соответствует выполнению операций слева направо, таким образом: `(3 * 4) / 2 = 6`.

## Оглавление:


1. Таблица приоритета и ассоциативности операций
2. Как возвести число в степень в C++?
3. Тест


## Таблица приоритета и ассоциативности операций

> ***Несколько примечаний:***
>
> - 1 означает самый высокий уровень приоритета, а 17 — самый низкий. Операции с более высоким уровнем приоритета выполняются первыми.
> - `L -> R` означает слева направо.
> - `R -> L` означает справа налево.

| Ассоциативность | Оператор | Описание | Пример |
|---|---|---|---|
| 1. Нет | `::` | Глобальная область видимости (унарный) | `::имя` |
| | `::` | Область видимости класса (бинарный) | `имя_класса::имя_члена` |
| 2. L -> R | `()` | Круглые скобки | `(выражение)` |
| | `()` | Вызов функции | `имя_функции(parameters)` |
| | `()` | Инициализация | `имя_типа(выражение)` |
| | `{}` | uniform-инициализация (C++11) | `имя_типа{выражение}` |
| | `type()` | Конвертация типа | `новый_тип(выражение)` |
| | `type{}` | Конвертация типа (C++11) | `новый_тип{выражение}` |
| | `[]` | Индекс массива | `указатель[выражение]` |
| | `.` | Доступ к члену объекта | `объект.имя_члена` |
| | `->` | Доступ к члену объекта через указатель | `указатель_объекта->имя_члена` |
| | `++` | Пост-инкремент | `lvalue++` |
| | `--` | Пост-декремент | `lvalue--` |
| | `typeid` | Информация о типе во время выполнения | `typeid(type)` или `typeid(выражение)` |
| | `const_cast` | Cast away const | `const_cast(выражение)` |
| | `dynamic_cast` | Type-checked cast во время выполнения | `dynamic_cast(выражение)` |
| | `reinterpret_cast` | Конвертация одного типа в другой | `reinterpret_cast(выражение)` |
| | `static_cast` | Type-checked cast во время компиляции | `static_cast(выражение)` |
| 3. R -> L | `+` | Унарный плюс | `+выражение` |
| | `—` | Унарный минус | `-выражение` |
| | `++` | Пре-инкремент | `++lvalue` |
| | `--` | Пре-декремент | `--lvalue` |
| | `!` | Логическое НЕ (NOT) | `!выражение` |
| | `~` | Побитовое НЕ (NOT) | `~выражение` |
| | `(type)` | C-style cast | `(новый_тип)выражение` |
| | `sizeof` | Размер в байтах | `sizeof(type)` или `sizeof(выражение)` |
| | `&` | Адрес | `&value` |
| | `*` | Разыменование | `*выражение` |
| | `new` | Динамическое выделение памяти | `new тип` |
| | `new[]` | Динамическое выделение массива | `new тип[выражение]` |
| | `delete` | Динамическое удаление памяти | `delete указатель` |
| | `delete[]` | Динамическое удаление массива | `delete[] указатель` |
| 4. L -> R | `->*` | Выбор члена через указатель | `указатель_объекта->*указатель_на_член` |
| | `.*` | Выбор члена объекта | `объект.*указатель_на_член` |
| 5. L -> R | `*` | Умножение | `выражение * выражение` |
| | `/` | Деление | `выражение / выражение` |
| | `%` | Деление с остатком | `выражение % выражение` |
| 6. L -> R | `+` | Сложение | `выражение + выражение` |
| | `—` | Вычитание | `выражение — выражение` |
| 7. L -> R | `<<` | Побитовый сдвиг влево | `выражение << выражение` |
| | `>>` | Побитовый сдвиг вправо | `выражение >> выражение` |
| 8. L -> R | `<` | Сравнение: меньше чем | `выражение < выражение` |
| | `<=` | Сравнение: меньше чем или равно | `выражение <= выражение` |
| | `>` | Сравнение: больше чем | `выражение > выражение` |
| | `>=` | Сравнение: больше чем или равно | `выражение >= выражение` |
| 9. L -> R | `==` | Равно | `выражение == выражение` |
| | `!=` | Не равно | `выражение != выражение` |
| 10. L -> R | `&` | Побитовое И (AND) | `выражение & выражение` |
| 11. L -> R | `^` | Побитовое исключающее ИЛИ (XOR) | `выражение ^ выражение` |
| 12. L -> R | `\|` | Побитовое ИЛИ (OR) | `выражение \| выражение` |
| 13. L -> R | `&&` | Логическое И (AND) | `выражение && выражение` |
| 14. L -> R | `\|\|` | Логическое ИЛИ (OR) | `выражение \|\| выражение` |
| 15. R -> L | `?:` | Тернарный условный оператор | `выражение ? выражение : выражение` |
| | `=` | Присваивание | `lvalue = выражение` |
| | `*=` | Умножение с присваиванием | `lvalue *= выражение` |
| | `/=` | Деление с присваиванием | `lvalue /= выражение` |
| | `%=` | Деление с остатком и с присваиванием | `lvalue %= выражение` |
| | `+=` | Сложение с присваиванием | `lvalue += выражение` |
| | `-=` | Вычитание с присваиванием | `lvalue -= выражение` |
| | `<<=` | Присваивание с побитовым сдвигом влево | `lvalue <<= выражение` |
| | `>>=` | Присваивание с побитовым сдвигом вправо | `lvalue >>= выражение` |
| | `&=` | Присваивание с побитовой операцией И (AND) | `lvalue &= выражение` |
| | `\|=` | Присваивание с побитовой операцией ИЛИ (OR) | `lvalue \|= выражение` |
| | `^=` | Присваивание с побитовой операцией «Исключающее ИЛИ» (XOR) | `lvalue ^= выражение` |
| 16. R -> L | `throw` | Генерация исключения | `throw выражение` |
| 17. L -> R | `,` | Оператор Запятая | `выражение, выражение` |

Некоторые операторы вы уже знаете из предыдущих уроков: `+`, `-`, `*`, `/`, `()`, `.`, `=`, `<` и `>`. Их значения одинаковы как в математике, так и в языке C++.

Однако, если у вас нет опыта работы с другими языками программирования, то большинство из этих операторов вам сейчас могут быть непонятны. Это нормально. Мы рассмотрим большую их часть на уроках этой главы, а об остальных расскажем по мере необходимости.

Эта таблица предназначена в первую очередь для того, чтобы вы могли в любой момент обратиться к ней для решения возможных проблем приоритета или ассоциативности.

## Как возвести число в степень в C++?

Вы уже должны были заметить, что оператор `^`, который обычно используется для обозначения операции возведения в степень в обычной математике, не является таковым в языке C++. В языке C++ это побитовая операция XOR. А для возведения числа в степень в языке C++ используется функция pow(), которая находится в **заголовочном файле** cmath:

```
#include <cmath>
...
double x = pow(3.0, 4.0); // 3 в степени 4
```

Обратите внимание, параметры и возвращаемые значения функции pow() являются типа double. А поскольку типы с плавающей точкой известны **ошибками округления**, то результаты pow() могут быть неточными (чуть меньше или чуть больше).

Если вам нужно возвести в степень целое число, то лучше использовать собственную функцию, например:

```
// Функция для возведения целых чисел в степень
int pow(int base, int exp)
{
    int result = 1;
    while (exp)
    {
        if (exp % 2)
            result *= base;
        exp /= 2;
        base *= base;
    }

    return result;
}
```

Не переживайте, если здесь что-то не понятно. Просто помните о проблеме переполнения, которая может произойти, если один из аргументов будет слишком большим.

## Тест

Из школьной математики нам известно, что выражения внутри скобок выполняются первыми. Например, в выражении `(2 + 3) * 4` часть `(2 + 3)` выполняется первой.

В этом задании есть 4 выражения, в которых отсутствуют какие-либо скобки. Используя приоритет операций и правила ассоциативности, приведенные выше, добавьте скобки в каждое выражение так, как если бы их обрабатывал компилятор.

> ***Подсказка:*** Используйте колонку *«Пример»* в таблице приоритета и ассоциативности операций, чтобы определить, является ли оператор **унарным** (имеет один операнд) или бинарным (имеет два операнда).

***Например***: `x = 2 + 3 % 4`

Бинарный оператор `%` имеет более высокий приоритет, чем оператор `+` или `=`, поэтому он выполняется первым: `x = 2 + (3 % 4)`. Затем выполняется бинарный оператор `+`, так как он имеет более высокий приоритет, чем оператор `=`.

**Ответ**: `x = (2 + (3 % 4))`.

Дальше нам уже не нужна таблица, чтобы понять ход обработки этого выражения компилятором.

**Задания:**

- *Выражение №1:* `x = 3 + 4 + 5`
- *Выражение №2:* `x = y = z`
- *Выражение №3:* `z *= ++y + 5`
- *Выражение №4:* `a || b && c || d`